

Machine Learning Report

Classification of SSH Connection Logs

Leïna Ben Bella
Cholé Daunus
Rania Azmane

Abstract

This report presents a complete machine learning workflow applied to SSH authentication logs. Two predictive tasks were addressed: binary classification of connection types (Normal vs. Suspect) and multiclass classification of failure types extracted from log messages. The project includes preprocessing, feature engineering, baseline modelling, hyperparameter optimisation, ensemble learning, PCA visualisation, correlation analysis, and tree-based interpretability tools such as Decision Trees, Random Forests and Gradient Boosting.

1 Introduction

SSH authentication logs contain a mixture of structured fields and unstructured text, making them challenging to use directly for machine learning. The goal of this project is to transform these logs into a structured dataset that allows accurate classification of connection behaviours. Two learning problems are investigated: (i) distinguishing normal SSH activity from potentially malicious connections, and (ii) predicting the failure category associated with each authentication attempt. These tasks form the basis of log-based anomaly detection systems widely used in cybersecurity.

Dataset limitations

Before entering the modelling stage, it is important to highlight a major limitation of this project: the dataset contains very few informative columns. Even after extracting additional features from the log content (such as `conn_per_hour`, `preauth`, or `Hour`), the overall feature space remains small. As a result, the separation between Normal and Suspect connections is almost entirely explained by only two or three variables.

This structural simplicity leads most models to reach extremely high accuracy very quickly. In practice, such “perfect” results do not necessarily reflect exceptional model performance, but rather the low complexity of the underlying data. This observation motivated the use of interpretability tools (heatmap, PCA, decision tree) and tree-based ensembles to understand whether more complex models could extract additional signal.

2 Data Preprocessing

The preprocessing phase aimed to transform the raw log data into a structured and clean dataset suitable for analysis and modeling. Initially, the Time column, when present, was

converted to a datetime format, allowing the extraction of the hour of each event into a new column, Hour. This temporal feature enables the computation of metrics such as the number of connections per IP per hour. In cases where the Time column was missing, a default value of -1 was assigned to the Hour column to avoid missing values.

Next, several features were extracted directly from the text content of each log entry. IP addresses were identified using a regular expression and stored in a dedicated column, with missing values replaced by "0.0.0.0". Usernames were similarly extracted by detecting the word following `user`, with unknown entries replaced by "unknown". Port numbers were captured from the text, converted to numeric values, and missing entries replaced by 0. Additionally, a boolean feature was created to indicate whether an event involved pre-authentication, based on the presence of the `[preauth]` tag.

Event classification was then performed to reduce the complexity of the data. Using the EventId column, events were labeled as either normal "N" or suspect "S" (cf documentation with information about each event). This approach reduced the number of categories from 26 to just 2, simplifying subsequent analysis and machine learning tasks.

Finally, unnecessary columns were removed from the dataset. While the original EventId was discarded after classification, the Content column was retained for reference despite being decomposed into more meaningful features. The final dataset thus included `ip`, `user`, `port`, `preauth`, `Hour`, `conn_per_hour`, `connection_type`, and `Content`.

Overall, this preprocessing pipeline effectively converted raw, unstructured log data into a clean, structured, and enriched dataset that supports both descriptive analysis and predictive modeling.

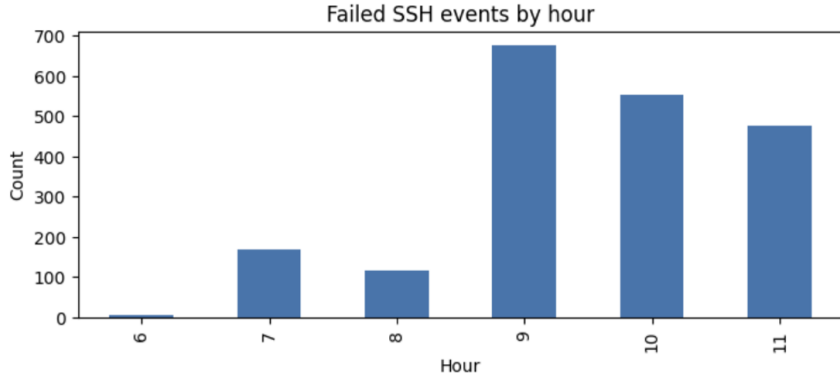


Figure 1: Failed SSH

3 Baseline Models

3.1 Binary Classification: Normal vs. Suspect

A Logistic Regression model was first trained to classify connections as Normal or Suspect. When using all extracted features, including `ip` and `user`, the model achieved extremely high accuracy. However, a closer inspection of these results revealed feature leakage:

some usernames and IPs appear exclusively in one class, making prediction artificially easy. Removing these features produced more realistic performance metrics.

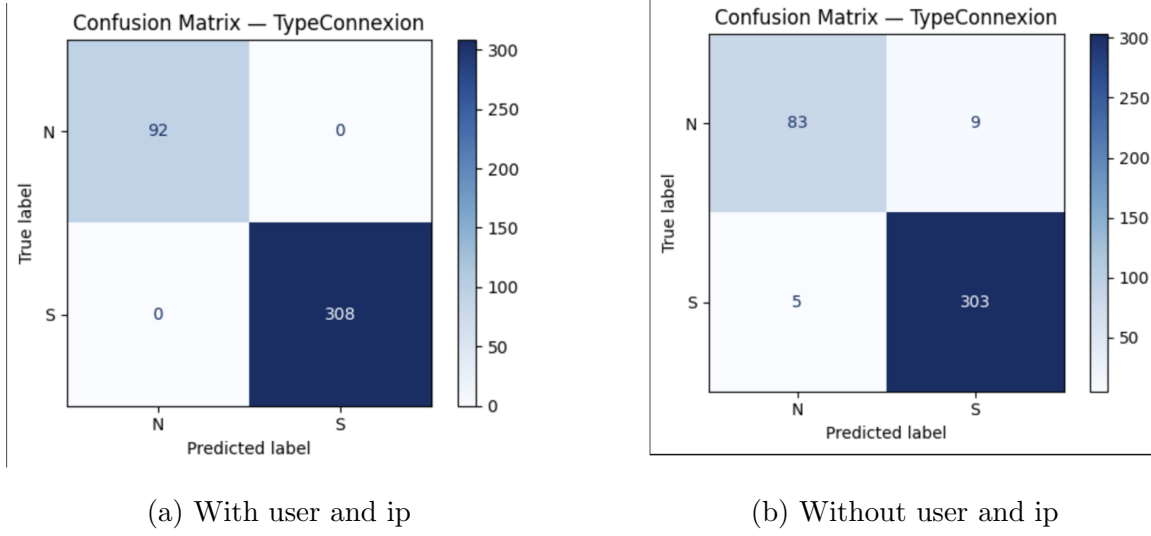


Figure 2: Effect of removing user/ip leakage.

Once identity-based features were removed, the Logistic Regression achieved approximately 96.5% accuracy. The confusion matrix shows that misclassifications occur mainly in borderline cases where behavioural patterns overlap between normal and malicious activity.

3.2 Failure Type Classification

A Linear SVM was trained to predict FailureType. Similarly to the previous task, removing the `user` variable improved the fairness of evaluation by eliminating leakage.

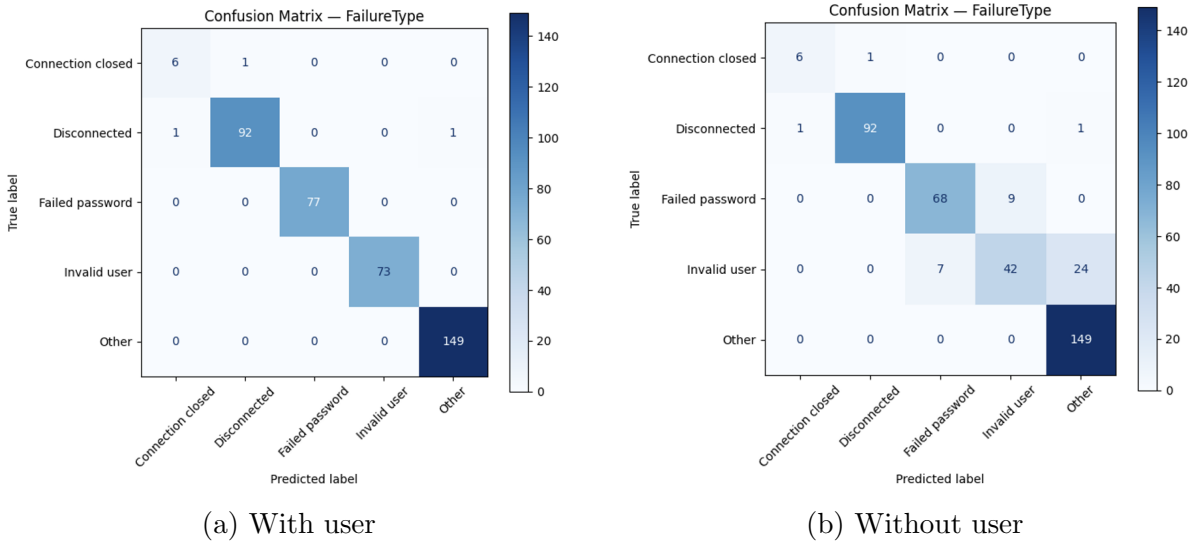


Figure 3: FailureType classification before/after removing user.

The SVM achieved strong performance on the most frequent categories but struggled with rare ones such as Connection closed. Misclassifications often occur between seman-

tically related categories, such as Failed password and Invalid user, reflecting the natural ambiguity present in repeated authentication attempts during attacks.

4 Grid Search Optimization

GridSearchCV was applied to both the FailureType and connection-type models to identify the best hyperparameters using cross-validation. For the FailureType model (Linear SVM), the best parameters were $C=1$ and $\text{max iter}=500$. The optimization did not improve performance, with overall accuracy remaining at 89%. This is likely due to the small, imbalanced dataset and limited lexical variability. Most classes were already well learned, except “Invalid user”, which remains difficult to predict without the user feature.

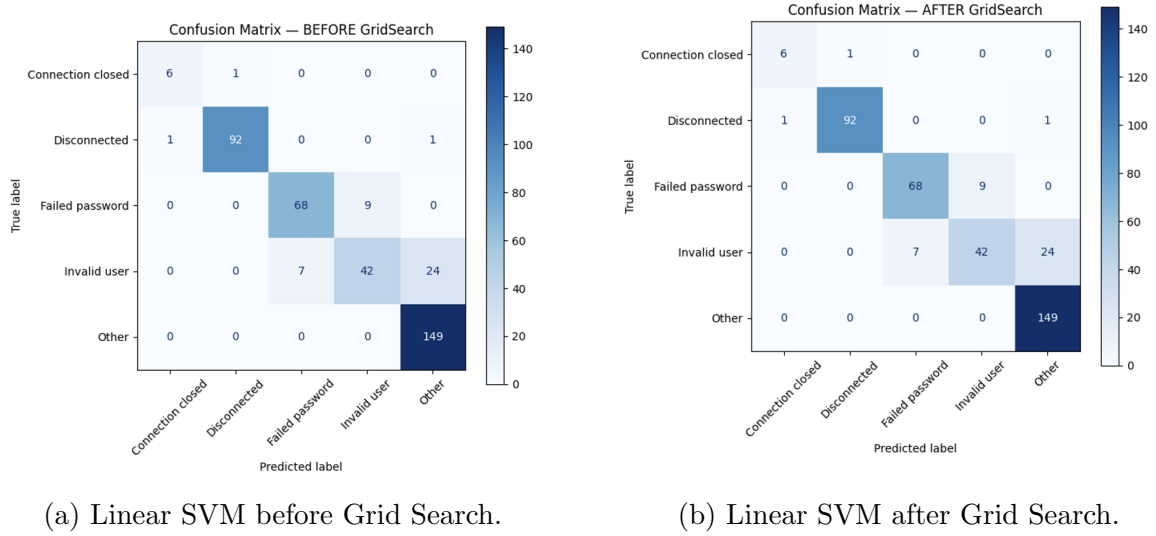
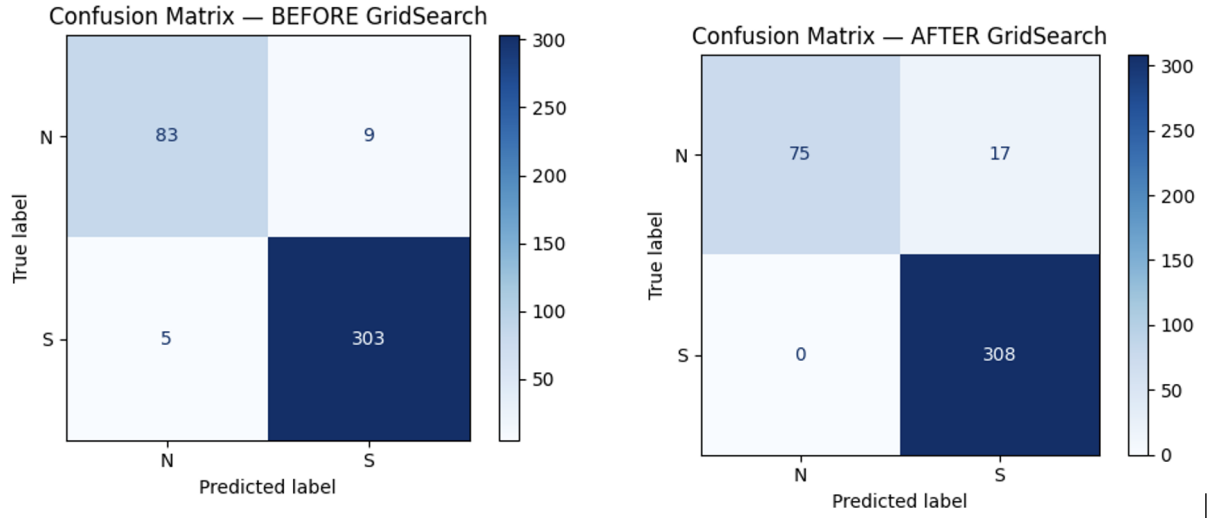


Figure 4: Confusion matrix BEFORE GridSearch and Confusion matrix AFTER GridSearch.

For the connection-type model (Logistic Regression), the best parameters were $C=0.1$ with $l2$ penalty. After optimization, recall for the suspect class reached 1.0, eliminating false negatives, but precision for the normal class dropped slightly, increasing false positives. Overall, total errors rose from 14 to 17. The optimized model became more sensitive to suspect connections but less precise for normal ones, illustrating a trade-off between class sensitivities.



(a) Logistic Regression before Grid Search.

(b) Logistic Regression after Grid Search.

Figure 5: Confusion matrix BEFORE GridSearch and Confusion matrix AFTER GridSearch

5 Correlation Heatmap

Because our dataset contains only a handful of numerical and boolean features, it was essential to verify whether these variables were redundant or highly correlated. A correlation heatmap was generated to quantify linear dependencies between the engineered features.

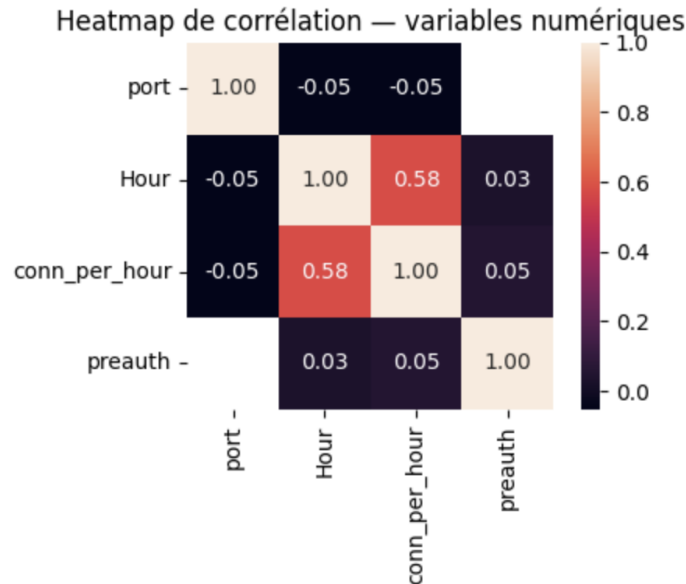


Figure 6: Correlation heatmap of numerical and boolean features.

The heatmap shows very limited correlation between variables, except for a moderate relationship between **Hour** and **conn_per_hour**. This confirms that each feature captures distinct information and that the small number of columns is the true bottleneck—not

multicollinearity. This analysis justifies why simple models already perform very well: the decision boundary is inherently simple.

6 PCA Visualisation

To visually inspect how the samples were organised in the feature space, a PCA projection onto two dimensions was performed. The PCA allows us to observe whether Normal (N) and Suspect (S) traffic naturally form separable clusters and where misclassifications occur.

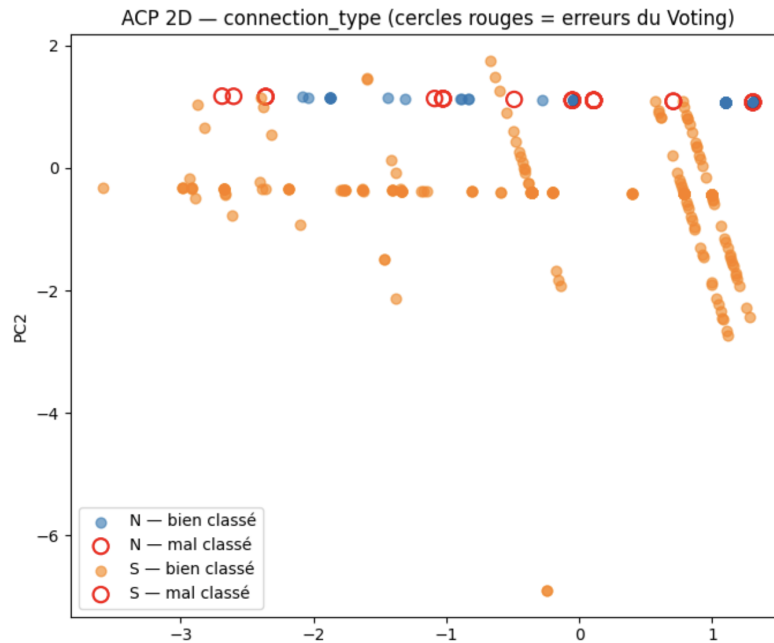


Figure 7: 2D PCA projection, with misclassified points highlighted.

The projection reveals that the two classes are mostly separable with only a small overlap region. Misclassified points lie exactly in this overlapping area, indicating that the observed errors are not due to poor model performance but to intrinsic ambiguity in the data itself. This confirms that increasing model complexity is unlikely to provide meaningful improvements.

7 Decision Tree Interpretation

A shallow Decision Tree (max depth = 3) was trained to obtain an interpretable model and understand the hierarchical structure of the decision process.

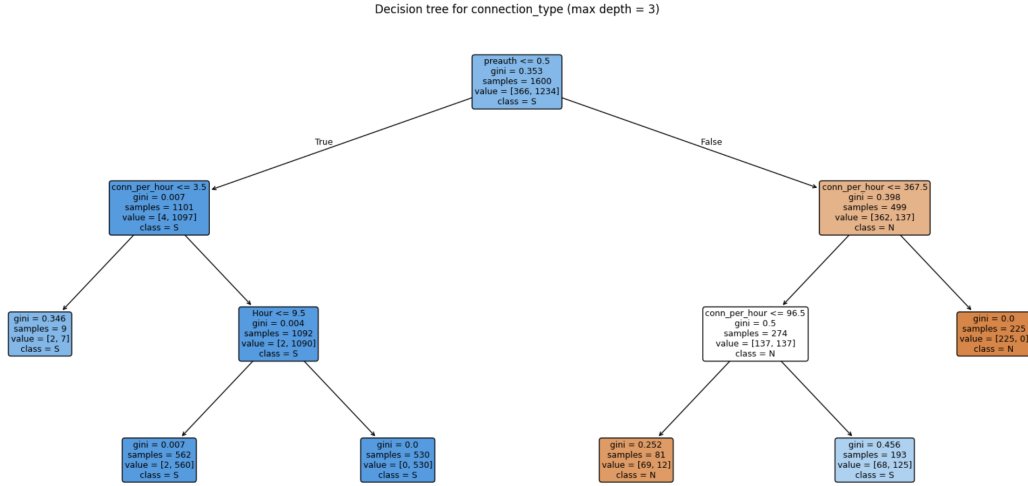


Figure 8: Decision tree for `connection_type`.

The tree reveals that the classification relies primarily on two variables: `preauth` and `conn_per_hour`. A single split on these features already separates the majority of Suspect connections from Normal ones. This confirms that the problem is linearly (and almost trivially) separable, which explains why Logistic Regression, SVM, Random Forest and Gradient Boosting all achieve similar results. The tree therefore validates our hypothesis that the dataset is simple rather than the models being exceptionally powerful.

8 Random Forest and Gradient Boosting

Ensemble tree-based models were also trained for comparison. The Random Forest classifier stabilises predictions by aggregating many decision trees trained on bootstrapped samples, while Gradient Boosting sequentially corrects previous errors.

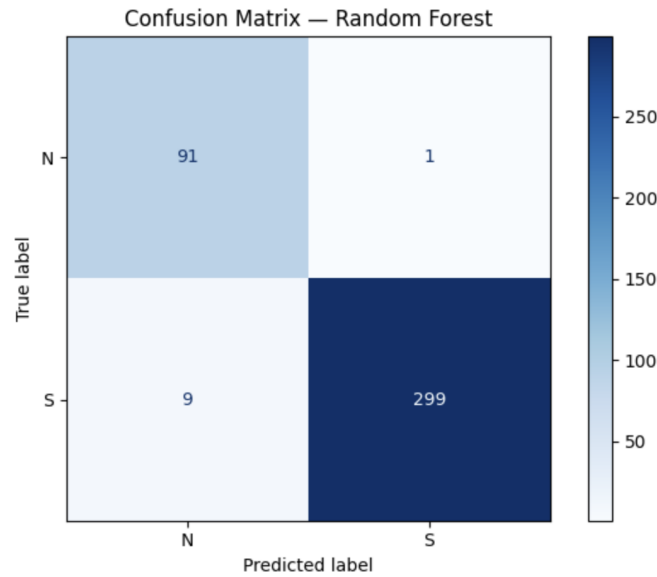


Figure 9: Random Forest confusion matrix.

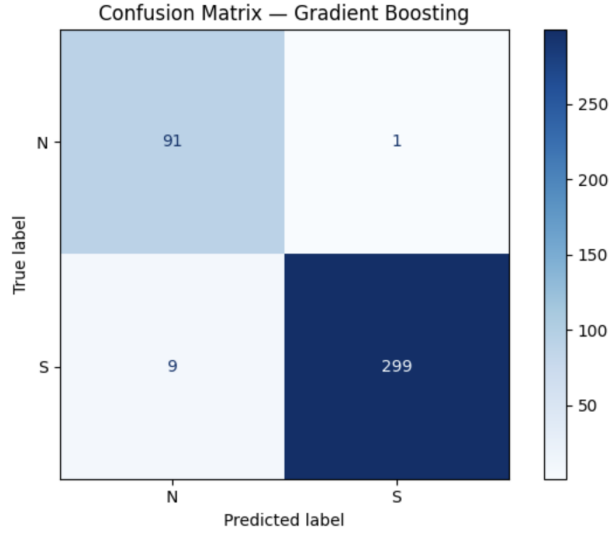
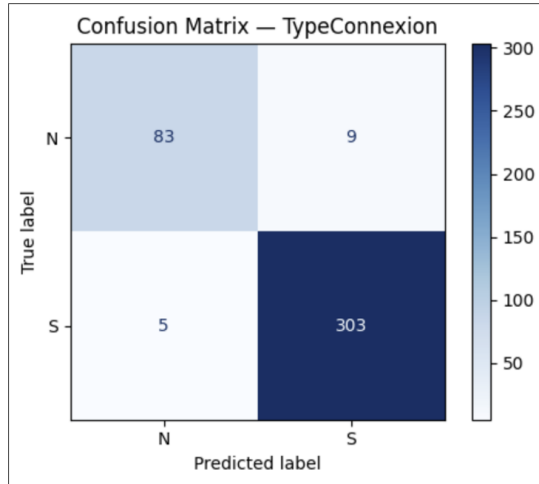


Figure 10: Gradient Boosting confusion matrix.

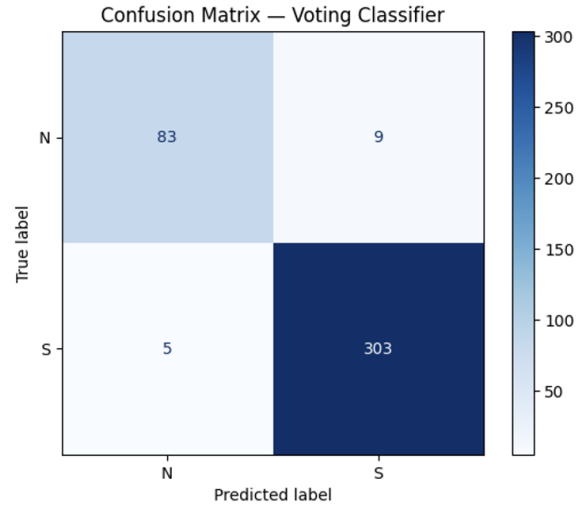
Both methods achieve similar performance to the logistic baseline, with Boosting offering slightly improved recall in certain cases. This similarity suggests that the engineered features already capture the essential signal, leaving limited room for non-linear models to extract additional structure.

9 Ensemble Modeling with Voting and Bagging

To improve connection type prediction, a Voting Classifier combining a Logistic Regression and a Linear SVM was used, with each base model wrapped in a BaggingClassifier. Bagging reduces variance by training multiple estimators on random subsets of the data, while Voting aggregates predictions from the different models. The ensemble achieved the same results as the baseline Logistic Regression, with 96.5% accuracy and similar precision and recall for both classes. This is expected because the dataset is relatively small and simple, and the features are already highly discriminative, making it difficult to improve performance. Furthermore, Logistic Regression is already a stable classifier, so applying Bagging does not significantly increase accuracy—it mainly helps reduce variance and improve robustness.



(a) Confusion matrix TypeConnexion



(b) Logistic Regression after Grid Search.

Figure 11: Confusion matrix TypeConnexion and Confusion matrix Voting Classifierh

10 Conclusion

This project highlights the entire pipeline from log preprocessing to machine learning evaluation. The main limitation of the dataset is its low feature dimensionality: even after engineering new variables, only a small set of features truly drives the classification. As a result, models achieve very high accuracy with little effort.

The heatmap confirmed that the features are not redundant. The PCA showed that classes are naturally separable. The decision tree provided a clear explanation of the model's behaviour. Random Forest and Gradient Boosting demonstrated that complex models do not improve the performance.

After these analyses, it became evident that retraining additional models would not lead to meaningful gains. The dataset's structure, not the algorithms, is the limiting factor. Future work should therefore focus on richer feature extraction (session duration, geographic origin, sequence models, temporal patterns) rather than on trying new classifiers.